indexes *(continued)*
mechanics of 158–160
process for 26–30
reindexing 190
using _bulk API 39–41
inverted 86–89
lifecycle management 227–234
lifecycle 228–229
lifecycle with rollover 231–234
managing manually 229–230
monitoring and managing 216–220
index statistics 217–218
multiple indexes and statistics 218–220
opening 211–212
overview 71–72, 193
reading 207–208
hidden indexes 208
public indexes 207–208
rolling over index aliases 224–227
searching across 310–312
shard sizing
setting up multiple indexes 476–477
setting up single index 475–476
shrinking 222–224
specifying analyzers for 261–263
field-level analyzers 262
index-level analyzers 262–263
splitting 220–222
templates for 212–216
creating component templates 215–216
creating composable templates 213–215
updating empty indexes 113
indexing API, explicit mapping 110–112
updating empty indexes 113
updating schema 112–113
indexOf method 173
IndexRequest object 538
indices attribute 482
indices namespace 534
indices_boost object 311
indices() function 535
Inference (inference) 461
Inflight requests circuit breaker 512
INFO level 488–489
infrastructure as code (IaC) tools 98
Ingest node role 84
ingest pipelines 523–530
loading PDFs into Elasticsearch 527–529
mechanics of 524–526
overview 523–524
ingest role 85, 524
initial_timeout attribute 492
inner hits object 291
integer_range data type 128
integer_range range 130
intersects (default) relationship 411
inverse document frequency (IDF) 90, 92, 302
inverted indexes 86–89
IP (internet protocol) adresses 130
ip data type 115, 130–131
ip_range 115, 128, 130, 452
iphones mapping 424
is_write_index property 204

J

JacksonJasonpMapper class 538
Java client 530–531
Java Concurrency in Practice (Bloch) 49–50
Java Performance (Oaks) 490
Java virtual machine (JVM) 490–491
JHLRC (Java High-Level REST client) 531
join data type 142–144
join query 327
joining_date attribute 112
JVM (Java virtual machine) 490–491

K

keep_words filter/list 261
key representation 455
keyword analyzer 252–253
keyword data types 118, 122–124, 316, 328, 431, 440
choosing over text types 501–502
constant_keyword type 123–124
fixing sorting using 108–109
keyword type 122–123
wildcard data type 124
keyword tokenizer 274
keyword types 122–123
Kibana 12
installing 520–522
downloading binary 521
installing via Docker 522
starting up on macOS 522
starting up on Windows 521
snapshots
automating 485–487
registering repositories 480–481
Kotlin Programming 347
KQL (Kibana Query Language) 352

L

lang parameter 177
language analyzers 256–258
large language model (LLM) 17
leaf queries 47, 296–297
less than (lt) operator 128, 323
less than or equal to (lte) operator 323, 325